Guía explicada del laboratorio de seguridad Umbral

Implementación peticiones demostraciones y correcciones

Trabajo Práctico Integrador de Seguridad en el Desarrollo de Software

Construimos una billetera ficticia en dos versiones y una guía de demostraciones. Esta edición ampliada explica el recorrido completo de las seis vulnerabilidades: qué hace la persona, qué envía el navegador, qué procesa el servidor, dónde se produce la falla y cómo actúa la corrección. Está pensada para estudiar el funcionamiento y poder explicarlo durante la exposición.

La versión vulnerable conserva seis fallas deliberadas. La corregida aplica los controles correspondientes sin eliminar las operaciones legítimas. Ambas usan el mismo diseño, cuentas ficticias y reglas de dinero; tienen bases y sesiones independientes. La corrida conservada reprodujo seis fallas y verificó seis defensas. Eso acredita esos escenarios y no demuestra que una aplicación sea completamente segura.

Cómo está organizada la explicación

Cada ejercicio tiene tres partes consecutivas. Primero se prepara y se ejecuta manualmente. Después se sigue la petición y el código vulnerable. Finalmente se estudian la corrección, los resultados y la verificación automática. Los ejemplos HTTP son representaciones simplificadas; los valores de cookies y tokens se reemplazan por marcadores explicativos.

Lectura	Qué permite entender
Aplicaciones y sesiones	Qué componentes existen y cómo se identifica una cuenta.
Herramientas del navegador	Cómo consultar la API y observar una petición real.
Autorización administrativa	Por qué una sesión válida no concede el rol administrador.
CSRF	Cómo otro origen provoca una operación con la sesión de la víctima.
XSS persistente	Cómo un concepto almacenado llega a ejecutarse al mostrarlo.
SQLi	Cómo una búsqueda cambia la consulta y expone movimientos ajenos.
Autenticación	Qué revelan los errores y cómo se limitan los intentos.
SSRF	Cómo el servidor consulta un servicio interno a pedido del usuario.
Automatización y exposición	Cómo interpretar evidencias y explicar el alcance real.

Fecha de esta ampliación: 9 de noviembre de 2026. El documento conserva la evidencia del 8 de noviembre; su redacción no implicó volver a ejecutar los ataques. Los recorridos manuales se contrastaron con el código y deben ensayarse en el equipo de exposición.

Aplicaciones y reglas que comparten las dos versiones

Qué se construyó

El frontend utiliza React, TypeScript, Vite y CSS propio. Flask sirve el frontend compilado y las rutas `/api`. Python implementa las operaciones y SQLite conserva usuarios, transferencias, ajustes y contadores de intentos. El modo se fija al iniciar cada proceso con `VARIANT`; una petición del navegador no puede cambiarlo.

Componente	Dirección que debemos utilizar
Billetera vulnerable	http://localhost:5000
Billetera corregida	http://localhost:5001
Guía y ejecutor	http://localhost:5002
Página atacante del ejercicio CSRF	http://localhost:5003

Cada versión permite registro, login, logout, saldo, transferencias, búsqueda de actividad, ajustes administrativos y cotizador. Cada instancia tiene base SQLite, almacenamiento de sesiones y nombre de cookie propios. El registro crea usuarios comunes con saldo cero; el reinicio restaura estas cuentas semilla:

Usuario	Contraseña ficticia	Rol	Saldo inicial
ana	Demo1234!	user	100000 centavos
bruno	Demo1234!	user	100000 centavos
admin	Demo1234!	admin	100000 centavos

Qué controles se conservan incluso en la vulnerable

Las contraseñas se procesan con hashes de Werkzeug. Los importes son enteros en centavos: `100` representa $1; `2500`, $25; `5000`, $50; `100000`, $1.000. El servidor valida importe positivo, destinatario existente, saldo suficiente y que emisor y receptor sean diferentes.

El débito, el crédito y el registro del movimiento se realizan en una transacción. El débito utiliza una condición de saldo suficiente. Esto impide que una transferencia normal quede parcialmente aplicada o que dos solicitudes gasten dinero que no existe. Las fallas deliberadas están acotadas a los seis escenarios descritos.

Cómo circula el tráfico local

El navegador llega a una pasarela que publica únicamente los puertos locales 5000 a 5003. Esa pasarela reenvía a destinos fijos de Docker. Las billeteras, la guía y los servicios simulados usan una red interna; el proveedor y el servicio interno no publican puertos propios.

Las peticiones pasan por esa pasarela antes de llegar a Flask. Los ejemplos siguientes muestran el intercambio visible para el navegador y omiten ese salto cuando no cambia la explicación. La configuración de red contiene el laboratorio; no se utiliza dinero ni información reales.

Sesiones cookies tokens y permisos

Cómo sabemos que una petición pertenece a Ana

Al iniciar sesión, el frontend obtiene primero `/api/csrf-token` y luego envía `/api/auth/login`. El servidor verifica el hash de la contraseña, limpia la sesión anterior, guarda `user_id` y la generación de datos y regenera el identificador de sesión. Flask-Session guarda el estado en el servidor mediante CacheLib.

El navegador recibe una cookie de sesión, llamada `wallet_vulnerable` o `wallet_secure` según la instancia. Su valor es un identificador de sesión protegido por el mecanismo utilizado; no enviamos el saldo ni un rol elegido por el cliente como fuente de autorización. En cada operación protegida, `current_user()` comprueba la generación de la sesión y consulta al usuario en SQLite.

```
uid = session.get('user_id')
if not uid:
    return None
# Se compara la generación de sesión con la generación de datos.
user = db.execute(
    'SELECT id,username,role,balance_cents FROM users WHERE id=?',
    (uid,)
).fetchone()
```

El fragmento abrevia `current_user` de `backend/app.py`. Si no hay usuario válido, `require_user()` devuelve 401. La sesión dura 30 minutos según la configuración y se invalida al cerrar sesión o restablecer datos. Un reset obliga a ingresar otra vez.

Por qué se puede consultar el token CSRF

`GET /api/csrf-token` entrega al frontend un token firmado ligado a su sesión. El frontend lo necesita para enviar mutaciones legítimas. No es un permiso administrativo ni una contraseña. El helper `walletPost` pide un token antes de cada POST; no conserva uno en un caché propio de React. Un token obtenido antes del login puede dejar de servir después de que se limpie la sesión, por eso el helper lo vuelve a consultar.

Elemento	Qué comprueba o permite
Cookie de sesión	Que el servidor pueda recuperar la sesión del navegador.
Credenciales de login	Que la persona conoce la contraseña de esa cuenta.
Token CSRF	Que la mutación incluye el token válido de esa sesión.
Rol consultado en servidor	Que la cuenta tiene permiso para esa operación.

Dos detalles del navegador que importan en el laboratorio

`localhost` y `127.0.0.1` son hosts diferentes para las cookies. Una sesión abierta con uno no acompaña automáticamente las peticiones dirigidas al otro. Utilizá siempre `localhost`; la pasarela además fija el encabezado Host a ese nombre.

Los puertos distinguen orígenes, pero no separan el alcance de las cookies del mismo host. Usamos nombres de cookie distintos para las dos billeteras. `HttpOnly` evita que JavaScript lea la cookie mediante `document.cookie`; el navegador sigue pudiendo enviarla. `SameSite=Lax` no bloquea por sí solo el caso entre puertos del mismo sitio localhost. Estas propiedades no equivalen a autorización de una operación.

Cómo observar y reproducir una petición con F12

Qué pestaña debe tener abierta la consola

Para ejecutar código con rutas relativas como `/api/me`, abrí F12 en la pestaña de la billetera que querés probar. En la vulnerable apunta a `localhost:5000`; en la corregida, a `localhost:5001`. Si lo pegás en la guía del puerto 5002, la ruta apunta a la guía y no a la billetera.

Podés consultar la cuenta abriendo `http://localhost:5000/api/me` en otra pestaña del mismo perfil. Es una petición GET. También podés ejecutar este bloque en la consola de la billetera:

```
{
  const response = await fetch('/api/me');
  console.log('HTTP', response.status);
  console.log(await response.json());
}
```

Las llaves crean un ámbito local y permiten repetir el bloque sin conflictos por variables `const` ya declaradas. No necesitamos Postman. Si se utilizara, habría que mantener su propia sesión, cookies y tokens; no hereda por sí solo la sesión del navegador.

Qué mirar en Red

1. Abrí F12 y seleccioná Network o Red. Activá Preserve log o Conservar registro para no perder solicitudes al navegar.
2. Limpiá el listado y realizá una sola acción del ejercicio. Usá el filtro por ruta para separar esa petición de las consultas periódicas.
3. En Headers, identificá Request URL, Request Method, Status Code, Origin y Content-Type.
4. En Payload, revisá el JSON o Form Data. En Response, revisá lo que respondió el servidor.
5. Compará después el saldo, los movimientos o el marcador que corresponda al caso. Un 200 no demuestra por sí solo que hubo una falla.

Un formulario HTML puede aparecer como documento o navegación, no como Fetch/XHR. En CSRF, utilizá el filtro All o Doc además de buscar `transfers`. Podés inspeccionar las cookies desde Application y desde la solicitud, pero no hace falta copiar su valor. Para la exposición, evitá proyectar identificadores de sesión o tokens completos.

Qué significa cada bloque copiable de la guía

Petición a observar describe una solicitud HTTP; no se pega literalmente en JavaScript. **Solicitud de ejemplo** del ejercicio de roles sí contiene un `fetch` ejecutable. **Concepto de prueba** se pega en la transferencia, **Búsqueda de prueba** en Actividad y **URL de servicio interno** en Cotizador. Autenticación utiliza el formulario de login y CSRF la página atacante.

Para una mutación normal verás normalmente un GET a `/api/csrf-token` y luego el POST. La función `walletPost` serializa JSON, agrega `X-CSRF-Token` y envía credenciales. La página atacante de CSRF omite deliberadamente ese paso del token.

Ejercicio 1 preparación y acción administrativa

Qué queremos demostrar

El administrador puede ajustar saldos y listar usuarios. Ana tiene rol común y no debería poder ejecutar esas funciones. La pregunta es si el servidor verifica ese permiso o se conforma con que exista una sesión. El caso demuestra falta de autorización por función, no un IDOR ni una evasión del login.

1. En `http://localhost:5002`, restablecé ambas versiones y verificá que las dos finalizaron correctamente.
2. Abrí `http://localhost:5000`, recargá si había quedado una pantalla anterior e ingresá como `ana` con `Demo1234!`.
3. Confirmá el saldo inicial de $1.000 y consultá `/api/me`. Debe indicar `username: ana`, `role: user` y `balance_cents: 100000`.
4. Abrí F12 en esa misma billetera y ejecutá el bloque siguiente. Solicitamos que una función administrativa acredite 5000 centavos a la cuenta de Ana.

```
{
  const me = await (await fetch('/api/me')).json();
  if (me.user?.username !== 'ana') {
    throw new Error('Ingresá como ana antes de continuar');
  }
  const csrf = await (await fetch('/api/csrf-token')).json();
  const response = await fetch('/api/admin/adjustments', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-CSRF-Token': csrf.token
    },
    body: JSON.stringify({
      user_id: me.user.id,
      delta_cents: 5000,
      reason: 'Demostración control de acceso'
    })
  });
  console.log('HTTP', response.status, await response.json());
}
```

1. Revisá la consola y el saldo. La vulnerable debería responder 200 con `ok: true` y mostrar $1.050. El ajuste queda en la tabla de ajustes; no debe confundirse con una transferencia en la lista de movimientos.
2. Abrí `http://localhost:5001`, ingresá como Ana y repetí exactamente el bloque. Debe responder 403 y mantener $1.000.

El bloque obtiene el identificador real de Ana; no hay que pegar el marcador `<id de ana>` del ejemplo HTTP. Las dos versiones usan bases independientes, por lo que el cambio de saldo en una no modifica la otra.

Ejercicio 1 petición y recorrido por el servidor

Qué envía exactamente la consola

El primer GET consulta el usuario y el segundo obtiene un token. El POST utiliza la sesión del navegador, un token válido y este cuerpo JSON. La representación omite encabezados que no son necesarios para explicar el caso:

```
POST /api/admin/adjustments HTTP/1.1
Host: localhost:5000
Origin: http://localhost:5000
Content-Type: application/json
X-CSRF-Token: <token de la sesión>
Cookie: wallet_vulnerable=<identificador de sesión>

{
  "user_id": 1,
  "delta_cents": 5000,
  "reason": "Demostración control de acceso"
}
```

El 1 representa el identificador observado de Ana en las semillas; el bloque ejecutable no depende de escribirlo a mano. `delta_cents` es la variación del saldo, no un saldo final. Se intenta sumar $50. La consola no manda un campo `role: admin`; la falla permite entrar con el rol común real.

Dónde se omite el permiso

En `backend/app.py`, tanto la ruta de ajustes como la de listado de usuarios llaman a `require_user(admin=True)`. La diferencia entre modos está dentro de esa función:

```
def require_user(admin=False):
    user = current_user()
    if user is None:
        abort(401)
    if admin and not teaching and user['role'] != 'admin':
        abort(403)
    return user
```

`teaching` es verdadero para la instancia vulnerable. Entonces `not teaching` es falso y se omite el rechazo por rol. La función devuelve a Ana aunque `admin=True`. La ruta continúa y valida los datos del ajuste: tipos enteros, variación no nula, razón de longitud permitida y saldo resultante no negativo.

El cambio se ejecuta mediante un UPDATE con parámetros y se guarda un registro en `adjustments`, dentro de una transacción. El SQL del ajuste no es la falla de este ejercicio. La falla ocurrió antes: un usuario sin permiso alcanzó una función administrativa válida.

Por qué usamos un token válido en el ataque

Ana puede consultar el token desde la propia billetera. Eso es normal y no le concede un rol. En la corregida queremos que la petición supere la protección CSRF y llegue a evaluar la autorización. Si omitiéramos el token, un 400 demostraría una petición mal preparada y no el rechazo del permiso administrativo.

Ejercicio 1 corrección y comprobación

Qué control agregamos y dónde actúa

En el modo corregido, `teaching` es falso. Si una ruta exige administración y el rol consultado en la base no es `admin`, `require_user` aborta con 403 antes de llegar al UPDATE. El manejador devuelve `{"error":"Acceso denegado"}`. No dependemos de que el frontend oculte el menú ni de lo que declare el cliente.

El frontend muestra Administración sólo al rol correspondiente, pero eso mejora la navegación; la protección real está en las rutas del servidor. La misma comprobación se aplica al GET de `/api/admin/users` y al POST de `/api/admin/adjustments`.

Comprobación	Vulnerable como Ana	Corregida como Ana
Sesión válida	Sí	Sí
Token válido del bloque	Sí	Sí
Rol real	user	user
Respuesta al ajuste	200 y ok true	403 y Acceso denegado
Saldo antes y después	100000 a 105000	100000 a 100000

Cómo demostrar que el administrador sigue funcionando

Después de la comparación, cerrá la sesión de Ana e ingresá como `admin`. Abrí Administración, seleccioná a Ana y aplicá un ajuste pequeño de +100 centavos; luego revertí con -100. Si la pantalla pide centavos, son 100 y -100, equivalentes a $1 y -$1. Revisá el formato que muestra el campo antes de enviarlo.

El caso automático utiliza una sesión independiente de administrador y verifica el +100. Después lo revierte para conservar un antes y después claro. Las dos versiones deben aceptar la operación legítima con 200. El bloqueo del usuario común no debe convertirse en un bloqueo de toda administración.

Qué mirar en F12 y en la evidencia

En Network buscá `admin/adjustments`. Mostrá el cuerpo, la respuesta y el estado. Consultá otra vez `/api/me` para demostrar que el saldo corresponde a Ana y que su rol sigue siendo `user`. El resultado automático compara el saldo antes y después y el control legítimo de administrador.

Si obtenés 401, falta una sesión válida o hubo un reset. Si obtenés 400 por token, repetí el bloque completo en la pestaña correcta. Si la corregida acepta la operación, verificá primero que no estés autenticado como admin y que el entorno realmente indique corregido.

Cómo explicarlo en la exposición

“Ana ya está autenticada, pero no tiene permiso administrativo. Hacemos una petición directa porque ocultar un botón no protege la API. La vulnerable sólo exige sesión y le permite sumar $50. La corregida consulta su rol en el servidor y rechaza el ajuste antes de modificar la base. El administrador legítimo sigue pudiendo operar.”

Ejercicio 2 preparación y página atacante CSRF

Qué queremos demostrar

Una persona puede tener una sesión abierta y visitar otra página que provoque una operación sobre la billetera. La página atacante prepara el destinatario y el importe; el navegador aporta la sesión si sus reglas de cookies lo permiten. El caso estudia la falta de comprobación de esa solicitud entre orígenes.

1. Restablecé ambas billeteras. Abrí `http://localhost:5000` e ingresá como Ana. Confirmá saldo de $1.000 y que `/api/me` reconoce a Ana.
2. En otra pestaña del mismo perfil, abrí `http://localhost:5003`. No uses una ventana incógnita separada ni mezcles localhost y 127.0.0.1.
3. Abrí F12 en la pestaña atacante, activá Preserve log y dejá el filtro All. Pulsá “Probar billetera vulnerable”.
4. Volvé a la billetera y comprobá el saldo de $975 y un movimiento de $25 hacia Bruno con el concepto indicado.
5. Para contrastar, restablecé, ingresá como Ana en `http://localhost:5001` y elegí “Probar billetera corregida” en el atacante. El saldo y la actividad deben permanecer sin cambios.

Qué hace el primer botón

En `lab/main.py`, `attacker_app()` sirve la página del puerto 5003. La opción que parece un botón es un enlace:

```
<a href="/?variant=vulnerable">Probar billetera vulnerable</a>
```

El clic genera primero `GET http://localhost:5003/?variant=vulnerable`. Esa petición no transfiere dinero. El servidor lee la variante, elige el puerto fijo 5000 y devuelve una segunda página con un formulario. Para `secure` elige 5001; no acepta destinos arbitrarios.

```
<form id="csrf-form"
      action="http://localhost:5000/api/transfers" method="post">
  <input type="hidden" name="recipient" value="bruno">
  <input type="hidden" name="amount_cents" value="2500">
  <input type="hidden" name="concept" value="Demostración CSRF">
  <button>Enviar demostración</button>
</form>
<script>document.getElementById('csrf-form').submit()</script>
```

Los campos ocultos fijan la operación. El script ejecuta `submit()` al cargar; por eso no hace falta pulsar después “Enviar demostración”. El formulario es HTML normal y provoca una navegación a la respuesta de la billetera, que puede mostrarse como JSON.

Ejercicio 2 petición cookies y recepción

Qué sale del navegador

El formulario envía datos de formulario, no JSON. No conoce la contraseña de Ana ni necesita escribir una cookie en JavaScript. Esta representación muestra la parte relevante de la solicitud:

```
POST /api/transfers HTTP/1.1
Host: localhost:5000
Origin: http://localhost:5003
Content-Type: application/x-www-form-urlencoded
Cookie: wallet_vulnerable=<identificador de sesión>

recipient=bruno&amount_cents=2500&concept=Demostraci%C3%B3n+CSRF
```

El navegador agrega la cookie conforme al destino y sus políticas. El código del atacante no usa `document.cookie`, no copia el identificador y no envía un campo que elija a Ana como remitente. La ruta de transferencias obtiene al emisor de la sesión recibida. Si la sesión fuera de otra cuenta, ésa sería la cuenta evaluada.

En este formulario el destinatario es siempre Bruno. Si quien está autenticado es Bruno, el servidor rechaza la operación por ser una transferencia a sí mismo. El mecanismo CSRF puede afectar otras cuentas, pero esta demostración está preparada para Ana como víctima. Para enseñar otra víctima habría que adaptar el destinatario de forma coherente; no se hizo en el caso actual.

Por qué esta cookie puede acompañar el formulario

Ambas páginas utilizan `http` y el host `localhost`. Los puertos diferentes crean orígenes diferentes, pero no sitios distintos para esta política. La cookie tiene `SameSite=Lax`, lo cual no bloquea este POST entre puertos del mismo sitio. En una página de un sitio externo real, las reglas SameSite pueden cambiar el resultado; no se debe generalizar esta prueba como si cualquier POST externo llevara siempre cookies.

`HttpOnly` impide leer el valor desde JavaScript, pero no impide que el navegador envíe la cookie. Además, las cookies no se aíslan por puerto: los servicios locales del mismo host no deben tratarse como dominios independientes para el secreto de cookies. Nuestra página atacante no las inspecciona ni las necesita; demuestra la ausencia de una defensa ante una petición de otro origen.

Qué hace la billetera vulnerable

`POST /api/transfers` llama a `require_user()`, que recupera a Ana mediante la sesión. `payload()` toma `request.form` porque el cuerpo no es JSON. El importe llega como texto `2500`, se convierte a entero y se validan destinatario, importe y saldo. Luego se realiza la transferencia atómica.

Como CSRF está desactivado en ese modo y no se rechaza Origin, el formulario llega a la operación y obtiene 200. El saldo de Ana baja 2500 centavos y se registra el movimiento. La aplicación comprobó una sesión válida, pero no exigió el token de protección para aceptar esa solicitud.

Ejercicio 2 corrección y comprobación

Cómo se protege la versión corregida

En backend/app.py, se registra CSRFProtect(app). La configuración desactiva su comprobación sólo cuando el modo es vulnerable. La ruta de token utiliza generate_csrf() y el frontend legítimo lo obtiene antes de cada POST:

```
const tokenResult = await walletRequest<{ token: string }>(
  "/api/csrf-token"
);
return walletRequest(path, {
  method: "POST",
  headers: { "X-CSRF-Token": tokenResult.data.token },
  body: JSON.stringify(body)
});
```

El helper de frontend/src/shared/api.ts también establece Content-Type para JSON y envía credenciales. Flask-WTF acepta el token en el encabezado o en el campo de formulario csrf_token. Consultarlo desde la propia billetera es parte del uso normal: el token no concede rol administrador.

La página atacante no consulta ese token ni lo incluye en el formulario. Como CSRFProtect se registra antes del chequeo propio de Origin, esta petición se rechaza primero por token ausente con 400. No llega al handler de transferencias. El mensaje personalizado es Token CSRF inválido o ausente.

Qué aporta la comprobación de Origin

En las mutaciones /api, el modo corregido también compara Origin con request.host_url cuando Origin está presente. Si no coincide, devuelve 403. Una petición con token válido pero origen distinto podría alcanzar este segundo control. La corrida del formulario sin token verificó el primer rechazo, no cada capa por separado. CORS no sustituye este control ni el token.

Observación conservada	Vulnerable	Corregida
Respuesta al formulario	200	400
Saldo Ana en centavos	100000 a 97500	100000 a 100000
Movimientos de Ana	0 a 1	0 a 0

Cómo comprobarlo y explicarlo

En Network observá el POST transfers, Form Data, Origin del puerto 5003 y la respuesta. Si sólo filtrás Fetch/XHR, la navegación del formulario puede no aparecer. El caso automático usa un mismo contexto de Chromium para víctima y atacante y comprueba saldo y movimientos antes y después.

Si devuelve Inicia sesión, comprobá que /api/me en localhost:5000 reconozca a Ana y que no haya ocurrido un reset. No basta con ver una pantalla vieja. Si la operación se hizo dos veces, puede haberse abierto dos veces el enlace: cada envío válido provoca un movimiento.

“El atacante prepara una operación y el navegador aporta la sesión de la víctima. La vulnerable la ejecuta. La corregida exige el token de esa sesión y rechaza el formulario que carece de él.”

Ejercicio 3 preparación del XSS persistente

Qué queremos demostrar

Una transferencia permite escribir un concepto que luego otra persona ve en Actividad. La falla aparece si el navegador interpreta ese contenido almacenado como HTML con instrucciones ejecutables. El emisor introduce el contenido y el receptor lo activa al visualizarlo: no necesita copiarlo en su consola.

En este caso la cuenta emisora es Ana y la receptora es Bruno. El efecto elegido es un marcador visual inocuo que dice “XSS ejecutado”. No extrae información ni envía datos a un tercero. El nombre persistente significa que el contenido queda guardado en la base y puede ejecutarse cuando se vuelva a representar.

Cómo realizar la demostración

1. Restablecé ambas versiones e ingresá como Ana en `http://localhost:5000`.
2. En Transferencias, escribí destinatario `bruno` e importe `1` en pesos. El frontend lo convertirá a `100` centavos.
3. Copiá exactamente el **Concepto de prueba** de la guía y pegalo en el campo de concepto. El valor completo también figura debajo; tiene 189 caracteres y entra en el límite de 200.
4. Enviá la transferencia y verificá que se registró. Cerrá sesión de Ana, ingresá como Bruno en la misma versión y abrí Actividad.
5. Observá el marcador. Repetí el recorrido en la corregida desde su estado inicial y comprobá que el concepto se muestra como texto.

```
<img src=x onerror="document.body.insertAdjacentHTML('afterbegin','<div data-testid=&quot;xss-
marker&quot; style=&quot;color:red;font-weight:bold&quot;>XSS ejecutado</div>');this.remove()">
```

Al copiar desde PDF puede introducirse un salto visual o un carácter de formato. Para ejecutar la demostración, preferí el botón Copiar de la guía o el texto de `docs/GUIA.md`. No agregues comillas externas al concepto ni lo pegues como un comando de consola.

Dónde debemos mirar

La observación principal es la Actividad de Bruno. La interfaz también puede renderizar el concepto en el resumen o en la actividad del emisor; por eso Ana podría ver el efecto antes. La prueba relevante conservada comprueba al receptor, con un contexto de navegador independiente en la automatización.

Dos pestañas del mismo perfil y la misma versión comparten la cookie. Si iniciás sesión como Bruno en una, también cambia la cuenta efectiva de la otra. Para mantener ambas cuentas abiertas al mismo tiempo, utilizá perfiles separados; si no, hacé logout y login de forma secuencial como indican los pasos.

Ejercicio 3 almacenamiento y ejecución del contenido

Primera petición la transferencia

La interfaz ejecuta `walletPost('/api/transfers', ...)`, obtiene un token normal y envía JSON. La siguiente representación abrevia el valor largo del concepto; no es un bloque para copiar literalmente:

```
POST /api/transfers HTTP/1.1
Host: localhost:5000
Content-Type: application/json
X-CSRF-Token: <token de sesión>
Cookie: wallet_vulnerable=<sesión de Ana>

{
  "recipient": "bruno",
  "amount_cents": 100,
  "concept": "<el concepto HTML completo de la página anterior>"
}
```

El servidor valida la operación como una transferencia normal. El concepto es una cadena y su longitud está permitida. Se guarda mediante un INSERT parametrizado junto con el emisor, el receptor y el importe. Los parámetros SQL evitan que el concepto altere esa sentencia, pero no determinan cómo lo mostrará después el navegador.

Segunda petición la actividad de Bruno

Cuando Bruno inicia sesión y abre Actividad, su navegador consulta `GET /api/movements`. El servidor devuelve JSON con el movimiento entrante y el concepto almacenado. La respuesta JSON no ejecuta el HTML por sí misma. El punto decisivo es cómo `MovementList` utiliza el valor al construir la página.

```
// Rama vulnerable en frontend/src/wallet/WalletApp.tsx
<span dangerouslySetInnerHTML={{
  __html: movement.concept || "Sin concepto"
}} />
```

`dangerouslySetInnerHTML` solicita a React que inserte el valor como HTML. El navegador crea un elemento `img` con origen `x`. Al fallar la carga de la imagen en el escenario previsto, se dispara el atributo `onerror`. Ese manejador ejecuta `document.body.insertAdjacentHTML(...)` y agrega un `div` con el marcador. `this.remove()` elimina la imagen que originó el error.

`"` representa comillas dentro del atributo HTML y permite construir los atributos del marcador. No buscamos leer cookies ni contraseñas. La prueba acredita ejecución de JavaScript y modificación visual del documento. No debe describirse como robo de sesión.

Por qué esto es persistente y dónde cruza el límite

El concepto nace como dato de un usuario, pasa por SQLite y JSON y finalmente se interpreta como código en el navegador de otro usuario. Haberlo guardado o haberlo recibido desde nuestra API no lo vuelve confiable. El límite que se cruza es entre contenido de usuario y HTML ejecutable de la aplicación.

Ejercicio 3 corrección y comprobación

Cómo se muestra el contenido en la corregida

En `MovementList`, el modo corregido devuelve el concepto como un valor de texto dentro de JSX. El esquema siguiente conserva la diferencia esencial de las dos ramas del componente:

```
<span className="movement-concept">
  {variant === "vulnerable" ? (
    <span dangerouslySetInnerHTML={{
      __html: movement.concept || "Sin concepto"
    }} />
  ) : (
    movement.concept || "Sin concepto"
  )}
</span>
```

En la segunda rama, React genera texto. Los caracteres `<img ...>` se ven como parte del concepto y no crean una imagen con un manejador. Se conserva la información escrita sin convertirla en instrucciones. No se intenta resolver el problema únicamente prohibiendo una palabra como script.

Qué aporta la política de contenido

La corregida también envía esta cabecera desde `security_headers` de `backend/app.py`:

```
Content-Security-Policy:
default-src 'self'; object-src 'none'; base-uri 'none';
frame-ancestors 'none'; form-action 'self'
```

La política agrega una defensa frente a contenido no autorizado, incluidos manejadores inline como el del ejemplo. La corrección principal de esta salida es renderizar como texto. La corrida completa verifica el resultado combinado del renderizado seguro y la política; no prueba cada capa por separado desactivando la otra.

Observación conservada	Vulnerable	Corregida
POST de transferencia	200	200
Saldo de Bruno	100000 a 100100	100000 a 100100
Concepto guardado	Sí	Sí
Marcador XSS en Bruno	Presente	Ausente
Concepto representado	HTML interpretado	Texto

Qué debemos verificar en F12

En Network, observá el POST con el concepto y luego el GET de movimientos desde Bruno. En Elements, el modo vulnerable crea un elemento con `data-testid="xss-marker"`; la corregida sólo contiene el texto del concepto. Un error de carga de la imagen es parte del disparador previsto y no significa por sí solo que la demostración falló.

La automatización transfiere mediante la interfaz y usa contextos distintos para emisor y receptor. Comprueba que la transferencia existe y que el marcador aparece únicamente en vulnerable. Si no aparece, comprobá el concepto completo, la cuenta receptora, la versión y que estés mostrando la actividad correspondiente.

“La transferencia funciona en ambas versiones. Lo que corregimos es la interpretación del concepto: el contenido del usuario deja de convertirse en HTML ejecutable cuando otra persona lo ve.”

Ejercicio 4 preparación de la búsqueda SQL

Qué queremos demostrar

Ana debe poder consultar las transferencias en las que participa, filtradas por concepto. No debe ver una transferencia de Admin a Bruno. El caso comprueba si un término de búsqueda puede modificar la consulta SQL y eliminar ese alcance de propiedad. Ana ya está autenticada: no es una inyección para entrar sin contraseña.

Preparación de los datos que hacen visible la falla

1. Restablecé ambas billeteras. En la vulnerable, ingresá como Ana y enviá $1 a Bruno con concepto `Compra propia`.
2. Cerrá sesión e ingresá como Admin. Enviá $1 a Bruno con concepto `Movimiento reservado`.
3. Cerrá sesión y volvé a ingresar como Ana. Abrí Actividad sin búsqueda. Debe verse Compra propia y no Movimiento reservado.
4. Pegá el siguiente término en el buscador. La interfaz consulta después de una breve pausa de 350 ms o al pulsar Buscar.
5. Repetí toda la preparación en la corregida y compará las filas devueltas.

```
' OR 1=1 --
```

Usá los nombres de cuenta en minúsculas como están en las semillas. No reinicies entre crear los dos movimientos y ejecutar la búsqueda, porque perderías los datos de preparación.

Qué petición envía la pantalla

El frontend agrega `q` a la ruta usando `encodeURIComponent`. La codificación permite transportar caracteres especiales en la URL; no constituye la defensa contra la inyección. La representación legible es:

```
GET /api/movements?q=' OR 1=1 -- HTTP/1.1
Host: localhost:5000
Cookie: wallet_vulnerable=<sesión de Ana>
```

En la petición real los espacios y otros caracteres se codifican. Por ejemplo, el valor puede verse como `'%20OR%201%3D1%20--`. Otros clientes pueden codificar también la comilla o usar + para espacios. Flask decodifica el parámetro y obtiene la misma cadena. Revisá Query String Parameters en Network para ver su valor interpretado.

Qué debemos obtener

En la vulnerable, el resultado incluye Movimiento reservado aunque Ana no es emisora ni receptora. En la corregida, el término se toma como texto literal y, con estos conceptos, devuelve una lista vacía. Limpiar la búsqueda debe volver a mostrar el movimiento propio en ambas versiones.

La lista vacía de la corregida no significa que desapareció el movimiento de Ana. Significa que ese concepto no coincide con el texto buscado dentro del conjunto permitido.

Ejercicio 4 cómo se modifica la consulta

Consulta que construye la versión vulnerable

`movements()` obtiene al usuario de la sesión y toma los primeros 200 caracteres de `q`. A continuación concatena una sentencia SELECT con una condición construida mediante una f-string. Este fragmento abrevia la selección de columnas y llama `uid` al identificador de Ana:

```
sql = select + (
    f"WHERE (t.sender_id={uid} OR t.recipient_id={uid}) "
    f"AND t.concept LIKE '%{q}%' ORDER BY t.id DESC"
)
rows = db.execute(sql).fetchall()
```

La intención es que el primer paréntesis limite los movimientos por propietario y que LIKE filtre por concepto. Para una búsqueda normal, por ejemplo `Compra`, la condición queda así:

```
WHERE (t.sender_id=1 OR t.recipient_id=1)
  AND t.concept LIKE '%Compra%'
ORDER BY t.id DESC
```

Qué cambia al introducir el término de prueba

Al interpolar la cadena `' OR 1=1 --`, la condición queda conceptualmente así:

```
WHERE (t.sender_id=1 OR t.recipient_id=1)
  AND t.concept LIKE '%' OR 1=1 --%' ORDER BY t.id DESC
```

La primera comilla cierra el literal que se estaba construyendo. `OR 1=1` agrega una condición verdadera. `--` comenta el resto de esa línea, incluidas las comillas que la aplicación esperaba añadir. Dada la precedencia entre AND y OR, la condición verdadera permite incluir también filas que no cumplen la restricción de Ana.

El contenido que debía ser un dato pasó a formar parte de la sintaxis SQL. El servidor ejecuta una consulta diferente de la que pretendía. La limitación a 200 caracteres no lo evita: la cadena utilizada es mucho más corta.

Cómo se comprueba la exposición

El SELECT une los identificadores de cuentas con sus nombres y devuelve los movimientos como JSON. El frontend representa la respuesta; no realiza después una nueva verificación de propiedad capaz de corregir lo que ya expuso la API. En el caso conservado aparecen Compra propia y Movimiento reservado.

Este escenario es una consulta de lectura. No ejecuta borrados, cambios de contraseña ni una secuencia de sentencias destructivas. La evidencia demuestra acceso a una fila ajena; no se debe atribuir al ejercicio un impacto que no se verificó.

El código usa `sqlite3.execute`, y otras operaciones de negocio están parametrizadas. No se debe concluir que toda la base está construida mediante concatenación porque se dejó este punto deliberado de enseñanza.

Ejercicio 4 corrección y comprobación

Qué cambia en el código corregido

El modo seguro conserva la estructura de la consulta y pasa los identificadores y el término como parámetros. El fragmento corresponde a `movements()` en `backend/app.py`:

```
rows = db.execute(
    select +
    'WHERE (t.sender_id=? OR t.recipient_id=?) '
    'AND t.concept LIKE ? ORDER BY t.id DESC',
    (user['id'], user['id'], '%' + q + '%')
).fetchall()
```

Cada `?` representa una posición para un valor. El motor recibe por separado la estructura SQL y esos valores. La cadena con comillas y OR ya no cierra un literal de la sentencia; se utiliza como el valor del patrón LIKE. El filtro de propiedad utiliza el identificador de la sesión y sigue presente en la consulta.

La solución no consiste en confiar en la codificación de la URL, en revisar sólo el frontend ni en reemplazar manualmente ciertas comillas. También sería insuficiente parametrizar el texto pero eliminar la condición de propiedad: ambos controles tienen una función diferente y deben conservarse.

Un matiz del patrón LIKE

Los caracteres `%` y `_` mantienen su significado de comodines dentro de LIKE. La parametrización no promete convertir todos los caracteres en una búsqueda literal exacta. Sí impide que el valor se vuelva sintaxis SQL y evita que evada la condición de propietario. Si se quisiera una búsqueda literal de comodines, sería un requisito adicional de búsqueda, no la misma corrección.

Observación conservada	Vulnerable	Corregida
Respuesta GET de búsqueda	200	200
Movimiento reservado en respuesta	Sí	No
Resultado para el término de prueba	Ambas transferencias	Lista vacía
Limpiar búsqueda	Movimiento propio	Movimiento propio

Qué observar y qué verifica la automatización

En Network filtrá `movements`, revisá `q` y abrí Response. Mostrá que el JSON vulnerable contiene `sender: admin`, `recipient: bruno` y el concepto reservado. Eso demuestra que el dato ya salió de la API, sin depender de cómo esté diseñada la pantalla.

El caso automático crea las dos transferencias con sesiones diferentes, consulta como Ana y busca el concepto reservado. En la segura exige que la respuesta sea 200, que no se exponga esa fila y que el resultado del término sea vacío. Las capturas muestran la misma búsqueda en la interfaz.

Si no hay diferencias, revisá primero que exista el movimiento Admin a Bruno y que hayas vuelto a entrar como Ana. Si el término produce un error SQL, hubo una alteración de la consulta, pero para mostrar la exposición prevista conviene revisar que el texto se haya copiado exactamente.

“La vulnerable inserta lo que escribimos dentro de la consulta y permite cambiar su lógica. La corregida separa los datos de la estructura SQL y mantiene fijo el alcance de la cuenta autenticada.”

Ejercicio 5 preparación del caso de autenticación

Qué queremos demostrar

La autenticación debe comprobar credenciales sin facilitar innecesariamente la identificación de cuentas y con controles de frecuencia. Aquí estudiamos dos comportamientos relacionados: mensajes que permiten enumerar usuarios y ausencia de limitación de intentos. No intentamos descubrir una contraseña real ni demostrar una toma de cuenta.

Ambas versiones guardan hashes. La comparación no depende de contraseñas en texto plano ni de una SQLi en el login. La cuenta conocida es ana; `noexiste` representa una cuenta ausente. La contraseña incorrecta de prueba es `Incorrecta123!`.

Cómo realizar la prueba manual

1. Restablecé ambas billeteras para limpiar los contadores. No inicies sesión exitosamente antes de esta comparación si querés reproducir los mismos conteos.
2. En el formulario de ingreso de la vulnerable, probá ana con `Incorrecta123!`. Anotá el mensaje y el estado 401 del POST.
3. Probá `noexiste` con esa misma contraseña. Compará el mensaje.
4. Hacé hasta seis intentos incorrectos adicionales sobre ana. Observá que la vulnerable no devuelve 429 por este límite.
5. Repetí en la corregida con sus contadores iniciales. Revisá el mensaje uniforme y la aparición de 429 en el bloque de intentos.

Qué petición genera cada clic en Entrar

Antes de cada POST, `walletPost` obtiene un token para esa sesión, incluso en la pantalla de login. La solicitud de credenciales es:

```
POST /api/auth/login HTTP/1.1
Host: localhost:5001
Content-Type: application/json
X-CSRF-Token: <token previo al login>
Cookie: wallet_secure=<sesión del navegador>

{"username":"ana","password":"Incorrecta123!"}
```

La petición para la cuenta ausente cambia `username` por `noexiste`. Es la misma contraseña incorrecta, para comparar la respuesta ante dos situaciones. En la vulnerable se cambia el puerto y el nombre de cookie, pero los campos son equivalentes.

Dónde mirar sin confundir errores

En Network filtrá `auth/login`. `401` indica credenciales rechazadas, `429` indica límite temporal y `400` por token indica que la petición no llegó a evaluar las credenciales. Un `401` inicial de `/api/me` al abrir el login es normal cuando todavía no hay una cuenta autenticada.

No uses Ana como si fuera necesariamente equivalente a ana: la búsqueda actual de nombres es sensible a mayúsculas. Las semillas utilizan minúsculas. El objetivo es comparar los nombres definidos por el caso, sin introducir otra variable.

Ejercicio 5 decisiones de login en el servidor

Recorrido de una solicitud válida en formato

`login()` toma el JSON, comprueba que usuario y contraseña sean cadenas y limita sus longitudes. En la corregida construye dos identidades de control: la dirección `request.remote_addr` y el nombre de cuenta convertido a minúsculas. Elimina registros anteriores a 60 segundos y cuenta los intentos de ambas identidades.

```
identities = (
    'ip:' + (request.remote_addr or 'unknown'),
    'account:' + username.lower()
)
# counts[0] corresponde a IP y counts[1] a cuenta.
if not teaching and (counts[0] >= 20 or counts[1] >= 5):
    return jsonify(error='Demasiados intentos'), 429, {
        'Retry-After': '60'
    }
```

Si ya se alcanzó alguno de los umbrales, rechaza la nueva solicitud antes de comprobar la contraseña. Un rechazo 429 no agrega otro intento. Si la solicitud sigue, registra un intento para la cuenta y otro para la IP. Se contabilizan los intentos admitidos por el control, incluidos los exitosos; un login exitoso no limpia el historial de esa ventana.

Qué información revela el modo vulnerable

La consulta del usuario está parametrizada en las dos versiones. La verificación usa el hash almacenado si existe la cuenta o un hash señuelo si no existe. Si las credenciales no son válidas, el modo de enseñanza diferencia estos errores:

```
if teaching:
    return jsonify(error=(
        'Contraseña incorrecta' if user else 'El usuario no existe'
    )), 401
return jsonify(error='Credenciales inválidas'), 401
```

Aunque ambos estados sean 401, el cuerpo permite saber si el nombre corresponde a una cuenta. Esa diferencia es la enumeración demostrada. La vulnerable tampoco registra ni aplica el límite del caso, de modo que las repeticiones continúan llegando a la comprobación de contraseña.

Qué ocurre si las credenciales son correctas

El servidor limpia la sesión, guarda el identificador del usuario y la generación de datos, marca la sesión como permanente y regenera su identificador. Devuelve los datos públicos de la cuenta. La interfaz consulta `/api/me` para mostrar el estado autenticado.

Este recorrido legítimo sigue existiendo en ambas versiones. Los límites pueden impedir temporalmente un login correcto después de muchos intentos, por eso antes de continuar otro ejercicio conviene restablecer o esperar a que venza la ventana. El objetivo es limitar solicitudes, no introducir una contraseña alternativa ni un acceso especial para las pruebas.

Ejercicio 5 corrección resultados y límites

Qué corregimos

Los errores por cuenta desconocida y contraseña incorrecta usan el mismo texto `Credenciales inválidas`. El hash señuelo evita omitir por completo el trabajo de verificación cuando falta el usuario. En este proyecto ese mecanismo de hash ya se utiliza en ambos modos; no debe atribuirse toda la diferencia a haberlo agregado sólo en la segura. Tampoco acredita tiempos idénticos para todas las rutas.

La corregida agrega un límite de cinco intentos por identidad de cuenta y veinte por IP en una ventana móvil de 60 segundos. Devuelve 429 con `Retry-After: 60`. El valor indica una espera conservadora fija; no se calcula en esta respuesta el número exacto de segundos que faltan para vencer el registro más antiguo.

Por qué el primer 429 no aparece en la sexta posición del reporte

El caso automático primero prueba una vez Ana con contraseña incorrecta y una vez noexiste. Después registra un bloque de seis intentos adicionales sobre Ana. Esa primera prueba de Ana ya consumió uno de sus cinco intentos permitidos.

Etapa sobre Ana	Intentos acumulados admitidos	Respuesta segura
Comparación inicial	1	401
Cuatro repeticiones siguientes	2 a 5	401 en cada una
Quinta repetición del bloque	Ya había 5	429
Sexta repetición del bloque	Continúa limitado	429

El bloque conservado es `[401, 401, 401, 401, 429, 429]`. En la vulnerable se obtienen seis 401. Además, los dos mensajes iniciales son distintos en vulnerable y uniformes en segura. Un ensayo manual con logins anteriores puede alcanzar el límite en otro momento.

Alcance real del control local

La cuenta se agrupa en minúsculas para el límite, pero la búsqueda SQLite del nombre es sensible a mayúsculas. Eso no convierte al login en indiferente a mayúsculas. El comportamiento de nombres y su normalización podría unificarse en un sistema de producción.

El límite por IP usa la dirección que Flask ve en `remote_addr`. Detrás de la pasarela puede ser la IP del proxy y agrupar varios navegadores. El proyecto no configura aquí una identidad de cliente distribuida ni una solución contra denegación de servicio. No debe presentarse como un control listo para cualquier despliegue.

Cómo explicarlo y comprobarlo

En F12 mostrás los dos cuerpos 401 y después la respuesta 429. El caso automático verifica mensajes uniformes y la presencia de una limitación en la corregida. No ejecuta un proceso de adivinación masiva ni demuestra acceso exitoso por fuerza bruta.

“La vulnerable informa qué cuentas existen y permite seguir intentando sin este límite. La corregida reduce esa información y restringe la frecuencia. Conservar hashes es necesario, pero no reemplaza estos controles del proceso de login.”

Ejercicio 6 preparación del cotizador SSRF

Qué queremos demostrar

El cotizador recibe una dirección y pide información desde el backend. La falla de SSRF consiste en poder inducir al servidor a consultar un destino diferente del proveedor previsto. En este laboratorio se demuestra exclusivamente con dos servicios simulados dentro de Docker; no se realizan consultas a sistemas externos ni a datos reales.

1. Restablecé ambas versiones, ingresá como Ana y abrí Cotizador en la vulnerable.
2. En URL del proveedor, consultá primero la dirección permitida. Debe aparecer una cotización USD.
3. Reemplazá la URL por la del servicio interno ficticio y pulsá Consultar.
4. Observá la respuesta `DATO INTERNO FICTICIO`. En F12 revisá el POST a la API de la billetera.
5. Repetí en la corregida: el proveedor legítimo debe funcionar y la URL interna debe rechazarse.

```
Proveedor legítimo simulado
http://quote-provider:8080/quote/USD

Servicio interno de demostración
http://private-ledger:8080/demo
```

Esos nombres sólo se resuelven dentro de la red del laboratorio. No hay que abrirlos en la barra de direcciones del navegador: el dato se pega en Cotizador y quien los consulta es Flask. La cuenta necesita una sesión válida para usar la función; SSRF no evita el login.

Primera petición navegador a billetera

La interfaz obtiene su token normal y envía el campo `url` como JSON:

```
POST /api/quotes/import HTTP/1.1
Host: localhost:5000
Content-Type: application/json
X-CSRF-Token: <token de sesión>
Cookie: wallet_vulnerable=<sesión de Ana>

{"url":"http://private-ledger:8080/demo"}
```

Esta es la petición que se ve en Network del navegador. Todavía no es la petición al servicio interno: sólo le pide a la billetera que importe información desde ese destino. El usuario controla la cadena `url` enviada en este primer salto.

Qué resultado buscamos comparar

La vulnerable admite el servicio interno ficticio y devuelve su mensaje. La corregida devuelve `Proveedor no permitido` sin hacer esa consulta. Para saber que no hubo una solicitud interna, el caso automático compara un contador del servicio además de observar el cuerpo de respuesta.

Ejercicio 6 recorrido de las dos solicitudes

Segundo salto billetera a servicio interno

En `quote()` de `backend/app.py`, el servidor recupera al usuario y la URL solicitada. En la vulnerable analiza esa URL y sólo permite los dos destinos de enseñanza. La condición incluye este fragmento:

```
contained = (
    parsed.scheme == 'http'
    and parsed.netloc in (
        'quote-provider:8080', 'private-ledger:8080'
    )
    and not parsed.username
    and not parsed.password
    and not parsed.fragment
)
```

Si no cumple, devuelve 400 por estar fuera del laboratorio. Si cumple, asigna `destination = supplied` y utiliza `requests` para hacer una solicitud HTTP desde el servidor. Con la URL del caso, el intercambio interno simplificado es:

```
GET /demo HTTP/1.1
Host: private-ledger:8080
```

El servicio ficticio devuelve:

```
{"message":"DATO INTERNO FICTICIO"}
```

La billetera interpreta ese JSON y lo envuelve en su propia respuesta:

```
{"result":{"message":"DATO INTERNO FICTICIO"}}
```

Qué cambia de identidad entre ambos saltos

El primer salto sale del navegador y utiliza la cookie para identificar a Ana ante la billetera. El segundo sale del contenedor de Flask y utiliza su acceso a la red interna. El código no reenvía al proveedor la cookie de Ana ni sus credenciales: crea un cliente de requests y hace el GET al destino.

El problema demostrado es utilizar la capacidad de red del servidor para llegar a un servicio que no corresponde al cotizador. No es una redirección del navegador ni una petición que la máquina del usuario haya realizado directamente al nombre interno.

Qué contiene la versión vulnerable y qué no

La excepción admite el proveedor y el servicio interno ficticio, con rutas y consultas en esos hosts. No admite cualquier URL de Internet o de la máquina anfitriona. El transporte común ya desactiva redirecciones y limita las respuestas en ambos modos. La falla intencional es que el cotizador acepta como destino otro servicio permitido por la contención general, aunque no sea el proveedor de cotización.

En F12 no verás el GET interno porque no sale del navegador. Verás el POST a la billetera y su respuesta. El contador del fixture y la lectura del código permiten explicar y comprobar ese segundo salto.

Ejercicio 6 corrección y comprobación

Cómo dejamos de aceptar un destino elegido libremente

En la corregida, el valor recibido debe coincidir exactamente con la URL canónica del proveedor. El destino efectivo se obtiene de la configuración confiable del servidor. No se construye copiando una dirección arbitraria de la petición:

```
PROVIDER = 'http://quote-provider:8080/quote/USD'
destination = app.config['PROVIDER_URL']
# En el modo corregido:
if supplied != PROVIDER:
    return jsonify(error='Proveedor no permitido'), 400
```

La URL interna no coincide y la ruta responde antes del GET. El chequeo es exacto: que una cadena contenga el nombre del proveedor no alcanza. La configuración `PROVIDER_URL` debe seguir siendo confiable; el cliente no la elige.

Cómo se limita la respuesta de un proveedor permitido

El cliente usa `trust_env=False` para no heredar proxies del entorno. Desactiva redirecciones con `allow_redirects=False`, pide respuesta en streaming y configura dos segundos de conexión y lectura. Exige estado 200 y Content-Type `application/json`.

Durante la lectura acepta como máximo 8192 bytes y comprueba un plazo de cinco segundos desde que comienza a consumir el cuerpo. No es una garantía de cinco segundos para toda la operación incluyendo resolución DNS, conexión y encabezados. El modo corregido además exige un objeto JSON con `currency: USD` y `rate_cents` entero positivo dentro del rango configurado.

Errores de transporte, tiempo agotado, contenido excesivo o respuesta inválida generan 502, `Proveedor no disponible`. Ese error no demuestra por sí solo un bloqueo SSRF correcto. Para el destino prohibido esperamos el rechazo previo y que el contador interno no cambie.

Observación conservada	Vulnerable	Corregida
Proveedor permitido	200 y cotización USD	200 y cotización USD
Petición de URL interna	200	400
Contador interno	0 a 1	1 a 1
Marcador interno en respuesta	Presente	Ausente

El valor inicial de la segura es 1 porque el servicio ficticio es compartido y la vulnerable ya lo consultó en ese par de pruebas. Lo importante es la diferencia: la segura no agrega una nueva llamada.

Cómo explicarlo y qué no debemos afirmar

El caso automático comprueba primero que el proveedor legítimo funciona, observa el contador antes y después y verifica el mensaje. La red y los nombres de Docker se consideran confiables en este escenario; no se implementa aquí una solución general frente a manipulación DNS ni autenticación TLS del proveedor ficticio.

“La vulnerable permite que el cotizador consulte otro servicio de la red interna. La corregida usa un destino fijo autorizado y valida la respuesta. Comprobamos que devuelve cotizaciones legítimas y que no consulta el servicio interno prohibido.”

Automatización evidencias y significado de los resultados

Qué ejecuta el laboratorio

El botón de cada caso inicia verificaciones en las dos instancias reales. Utiliza Chromium sin ventana propia, formularios de la interfaz y solicitudes autenticadas a las API públicas según la necesidad del caso. Las sesiones automáticas son distintas de las humanas, pero los datos se guardan en las mismas bases visibles.

En CSRF se mantiene la sesión de víctima y atacante en un mismo contexto de navegador. En XSS se separan los contextos de emisor y receptor. Roles comprueba también al administrador legítimo. SQLi prepara dos movimientos. Autenticación compara errores y repite intentos acotados. SSRF verifica proveedor, respuesta y contador interno.

El lote completo realiza doce verificaciones secuenciales y restablece las billeteras antes de cada ejercicio. Eso invalida las sesiones humanas al avanzar. Para observar cambios en vivo es más claro ejecutar un caso individual después del reset e iniciar sesión otra vez como observador. Las pantallas autenticadas consultan saldo y movimientos cada dos segundos mientras están visibles.

Resultado que ya se conservó

La corrida `000c0cb404874efcbacac65a876213c1`, del 8 de noviembre de 2026, finalizó con doce resultados esperados. El registro completo está en `evidencias/ejecucion-final/result.json`, junto a las capturas por caso y versión. Esta ampliación explica esos datos existentes; no presenta una nueva corrida.

Caso	Efecto vulnerable	Defensa verificada
Roles	Ana suma 5000 centavos	403 y saldo sin cambios
CSRF	Transferencia de 2500	400 y ningún movimiento
XSS	Marcador en Bruno	Texto sin marcador
SQLi	Fila Admin a Bruno expuesta	Sin fila ajena
Autenticación	Errores distintos sin límite	Error uniforme y 429
SSRF	Consulta interna y mensaje	Sin nueva consulta interna

Cómo leer los estados

`attack_reproduced` acredita el efecto vulnerable esperado. `blocked` es la defensa comprobada para ese caso, incluso cuando la respuesta HTTP es 200. `unexpected` indica una diferencia frente al resultado previsto. Una excepción, servicio inaccesible, timeout o precondición alterada puede producir un resultado inconcluso. Una cancelación tampoco acredita una defensa.

La guía conserva las últimas diez ejecuciones en almacenamiento separado del reset y muestra solicitudes resumidas sin contraseñas, cookies ni tokens. La copia exportada final permanece en la carpeta de evidencias. Cancelar detiene trabajo pendiente, pero no revierte operaciones que ya ocurrieron. Un estado completed sólo describe que terminó la ejecución; hay que revisar los resultados de cada caso.

Problemas frecuentes durante una demostración

Inicia sesión aunque hay una billetera abierta

Consultá `/api/me` en el host y puerto exactos del destino. Una pantalla puede conservar datos anteriores al reinicio. Un reset invalida las sesiones. Además, `localhost` y `127.0.0.1` tienen cookies diferentes. Para CSRF deben coincidir el perfil, el host y la sesión de la víctima; iniciá sesión de nuevo en localhost y abrí después el atacante.

Origen no permitido en la corregida

La pasarela fija Host como `localhost:<puerto>`. Si navegás por `127.0.0.1`, el navegador puede enviar ese nombre en Origin y no coincidir con lo que Flask considera su origen. Utilizá `http://localhost:5001`, recargá e ingresá otra vez. Recargar puede limpiar una pantalla anterior, pero no debe presentarse como sustituto de corregir un nombre de host distinto.

Token CSRF inválido o ausente

El token y la cookie deben pertenecer a la misma sesión. Repetí el bloque completo desde la billetera o utilizá el formulario normal. No reutilices un token anterior a un login, logout o reset. El helper obtiene uno antes de cada POST. Para probar roles, el token debe ser válido; para demostrar CSRF, el formulario atacante lo omite deliberadamente.

Cambiar de cuenta en dos pestañas cambia ambas

Las pestañas del mismo perfil comparten la sesión de esa billetera. Para Ana y Bruno simultáneos usá perfiles separados. Para CSRF, al contrario, atacante y víctima necesitan el mismo perfil para que el navegador tenga la sesión que se pretende demostrar.

El marcador XSS no aparece o la SQLi no expone filas

En XSS, comprobá el concepto completo, la variante y la actividad de Bruno. En SQLi, comprobá que creaste los dos movimientos y volviste a ingresar como Ana. Sin un movimiento ajeno, no hay contraste observable aunque la consulta sea vulnerable. El formulario de búsqueda dispara consultas después de 350 ms; esperá o pulsá Buscar.

El login limita antes de lo esperado

Los intentos anteriores y los exitosos también pueden consumir cuota. La IP que ve Flask puede ser la del proxy y agrupar clientes. Restablecé antes del caso y registrá los códigos que realmente recibiste. Un 429 no significa contraseña incorrecta: significa que la frecuencia está limitada.

El cotizador falla o el ejecutor está ocupado

No abras los nombres internos desde la barra del navegador. Usalos en Cotizador. Un 502 no demuestra una defensa SSRF: puede ser un proveedor caído o una respuesta inválida. Si la guía indica una operación activa, esperá o cancelá; si el reset fue parcial, repetilo hasta completar ambas versiones antes de iniciar otra prueba.

Regla para todas las demostraciones: anotá la precondición, la petición y el cambio efectivo. Si faltan datos para atribuir el resultado al control estudiado, informalo como inconcluso y prepará de nuevo el ejercicio.

Guion para explicar el trabajo práctico

Apertura y recorrido

“Construimos una billetera ficticia con dos versiones comparables. Las operaciones legítimas y el diseño son equivalentes. En una dejamos seis fallas acotadas y en la otra aplicamos los controles. Una tercera aplicación permite preparar las demostraciones, ejecutarlas y conservar evidencias.”

1. Mostrá las cuatro direcciones y las cuentas ficticias. Explicá centavos, bases independientes y reset.
2. Mostrá un login y una operación normal. Ubicá en F12 el token, el POST y la respuesta.
3. Elegí un ejercicio y explicá su precondición antes de ejecutarlo. No presentes un error aislado como demostración.
4. Mostrá la entrada o solicitud, el efecto en vulnerable, la línea de código que lo permite y el contraste corregido.
5. Cerrá cada caso comprobando que la operación legítima sigue disponible. Después revisá una evidencia del ejecutor.

Qué límite de confianza cruza cada caso

Caso	Dato o acción que cruza el límite	Control relevante
Roles	Un usuario común alcanza una función administrativa	Autorización del rol en servidor
CSRF	Otro origen induce una mutación con sesión ajena	Token y origen
XSS	Texto de usuario se convierte en HTML ejecutable	Salida como texto y CSP adicional
SQLi	Texto de búsqueda pasa a formar parte del SQL	Parámetros y propiedad
Autenticación	Respuesta revela cuenta y admite repeticiones	Error uniforme y frecuencia
SSRF	Una URL de entrada elige un destino del servidor	Proveedor fijo y límites

Preguntas que deberíamos poder responder

Por qué una cookie válida no basta. Identifica una sesión, pero no demuestra que la persona quiso realizar una operación ni que tenga permiso administrativo. Se requieren controles distintos para CSRF y roles.

Por qué CSRF no necesita robar cookies. La página induce una solicitud y el navegador puede adjuntar las cookies del destino. La demostración no lee su valor. Las políticas de cookies determinan si acompañan esa solicitud concreta.

Por qué XSS y SQLi pueden responder 200 en la corregida. La transferencia sigue siendo válida y una búsqueda puede no encontrar coincidencias. Lo que cambia es la ejecución del concepto o la exposición de filas, no necesariamente el estado HTTP.

Por qué el servidor debe validar aunque el frontend lo haga. Una petición puede enviarse sin usar los botones previstos. El cliente mejora la experiencia, pero no concede permisos ni reemplaza las reglas del backend.

Qué prueba la automatización. Repite escenarios definidos con comprobaciones y evidencia. No revisa por sí sola todas las rutas ni acredita ausencia de vulnerabilidades desconocidas. La explicación del código y el ensayo manual siguen siendo necesarios.

Alcance pendientes y referencias para seguir estudiando

Qué se verificó y qué no

La evidencia del proyecto incluye la corrida de doce contrastes esperados, 21 pruebas de backend y guía, compilación y comprobación de TypeScript, y revisión de interfaz en escritorio y a 390 px. Las pruebas de CSRF y XSS utilizaron Chromium. Este documento amplía la explicación de esas verificaciones existentes; no reemplaza ejecutar los controles de nuevo después de futuros cambios.

La versión corregida resuelve los seis escenarios diseñados. Usa HTTP local, sin cifrado de transporte y sin cookies Secure para ese entorno. El almacenamiento de sesiones en archivos, los límites por cuenta e IP y el proceso de despliegue son didácticos. Las revisiones de dependencias de Python y frontend no abarcan por sí solas el sistema operativo de las imágenes ni todo el navegador.

Hay un pendiente conocido en `lab/runner.py`: si falla la escritura del estado de una ejecución en ciertos puntos, el bloqueo puede quedar ocupado. Requiere liberar siempre el bloqueo ante ese error. No afecta el registro exitoso conservado, pero impide afirmar que el ejecutor recupera cualquier fallo de almacenamiento.

El reset de ambas bases no es una transacción distribuida. Cada instancia restaura transaccionalmente sus datos y la guía exige completar ambas antes de continuar si hay un fallo parcial. También queda por ensayar todo el recorrido manual y comprobar el arranque sin Internet en el equipo de exposición, con imágenes y dependencias ya preparadas.

Archivos para contrastar la explicación

Las rutas son relativas a `tpi-billetera`: `backend/app.py` contiene las decisiones del servidor; `frontend/src/shared/api.ts`, los helpers de peticiones; `frontend/src/wallet/WalletApp.tsx`, los formularios y el renderizado; `lab/main.py`, la página atacante; `lab/api_cases.py` y `tests_browser/scenarios.py`, las comprobaciones; `compose.yaml` y `backend/gateway.py`, la red y pasarela. Los resultados están en `evidencias/ejecucion-final/`.

Los fragmentos se presentan completos o abreviados según su finalidad y se indica cuando son esquemas. Los ejemplos HTTP omiten encabezados no relevantes y utilizan marcadores para sesiones y tokens. No deben pegarse como comandos en la consola. El bloque ejecutable de roles y los valores de prueba sí se copian en el lugar indicado.

Lecturas técnicas de apoyo

La implementación del proyecto es la fuente de sus resultados. Las siguientes referencias explican los mecanismos generales y no certifican el laboratorio:

- MDN sobre cookies y sus atributos: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies
- MDN sobre CSRF y límites de confianza: https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/CSRF
- Flask-WTF sobre CSRFProtect tokens y errores: https://flask-wtf.readthedocs.io/en/1.2.x/csrf/
- Python 3 sobre sqlite3 y sustitución de parámetros: https://docs.python.org/3.13/library/sqlite3.html

Para justificar cada corrección, relacioná siempre la entrada, la petición, la decisión del código y el estado resultante. Si una comprobación no se realizó o una capa no se probó de forma aislada, indicá ese límite en lugar de atribuirle un resultado no observado.